Proyecto Integrador

Desarrollo en la Nube

Otoño 2026

Prof. Marcela Rosales, ITESO

---

1 Descripción del proyecto integrador

El objetivo del proyecto integrador es que los alumnos usen de forma integral los servicios y herramientas vistas durante el semestre, así como que tomen decisiones técnicas para desarrollar un producto tecnológico en la nube.

No es suficiente que los alumnos desarrollen código funcional, también deben poder explicar las decisiones técnicas que tomaron, las problemáticas a las que se enfrentaron y cómo las resolvieron. La capacidad de razonar y defender esas decisiones es parte central de la evaluación.

El proyecto integrador se conforma de tres fases: en la primera se plantea el proyecto que se va a desarrollar, en la segunda se presentan avances y en la tercera se muestra la aplicación funcional en la nube.

2 Equipo de trabajo

El proyecto se realiza en equipos de 3 personas, cada integrante del equipo debe asumir una responsabilidad durante el desarrollo del proyecto, sin embargo todos los compañeros deben conocer el proyecto completo a nivel general. Es válido que alguien profundice en su área, pero nadie puede desconocer lo que hace el resto del equipo.

También es importante mencionar que colaborar no está reñido con tener roles definidos. Si un integrante encuentra dificultades en sus actividades, se espera que el equipo lo resuelva en conjunto. Los roles existen para dar claridad sobre las responsabilidades, no para aislar a las personas.

Roles

- Backend flujo end-to-end 1 + Frontend
- Backend flujo end-to-end 2 + CI/CD
- Backend flujo end-to-end 3 + Monitoreo

3 Requisitos del proyecto y complejidad

1. **Incorpora el uso de al menos 5 servicios de AWS vistos en clase:** por ejemplo:
   - Cómputo: EC2, Lambda.
   - Almacenamiento: S3.
   - Bases de datos: RDS, DynamoDB.
   - Mensajería: SQS, SNS.
   - Seguridad: IAM, Secrets Manager.
   - Serverless: API Gateway, Step Functions.
   - Observabilidad: CloudWatch.
2. **Desarrollar 3 flujos end-to-end:** cada flujo incluye validación del input del usuario, lógica de negocio y entrega de un resultado observable. Un flujo end-to-end es una secuencia de acciones que comienza con una entrada del usuario y termina con un resultado concreto. Por ejemplo, un archivo generado, un mensaje enviado o un registro actualizado. Al menos uno de estos flujos debe aprovechar características propias de la nube que serían difíciles de replicar en un entorno local.
3. **Repositorio de GitHub:** El repositorio es evidencia del trabajo en equipo durante el semestre, todos los integrantes del equipo deben tener participación en el repositorio con el historial de commits. El repositorio debe tener todo el código, scripts e instrucciones para que el proyecto pueda reproducirse.
4. **Demo:** Como evidencia para la revisión de la fase 2 se debe entregar un video con un demo parcial de avances con una duración máxima de 10 minutos. Se debe hacer énfasis en los avances, es decir lo que se trabajó entre fases y lo que falta para terminar el proyecto. Para la fase 3 el equipo debe mostrar su proyecto completo en una presentación de 20 minutos.
5. **Diagrama de arquitectura:** El equipo debe entregar un diagrama que muestre la arquitectura del proyecto, los servicios utilizados y cómo se relacionan entre sí. Se recomienda Excalidraw o draw.io.
6. **Frontend:** El proyecto debe incluir un frontend. No hay restricción de lenguaje ni framework. La solución debe correr en la nube.
7. **Backend:** El proyecto debe incluir un backend, no hay restricción de lenguaje ni framework. La solución debe correr en la nube.
8. **CI/CD:** El proyecto debe tener un pipeline de CI/CD el cual se debe implementar con GitHub Actions. Cuando se integre código en la rama principal, el workflow debe ejecutarse automáticamente y desplegar los cambios en la nube. Se espera que el pipeline incluya pruebas unitarias como parte del proceso de integración continua.
9. **Monitoreo:** El proyecto debe incluir un sistema de monitoreo el cual tenga 5 métricas relevantes para la aplicación, 2 alertas que se disparen cuando una métrica supere un umbral definido y 1 dashboard en CloudWatch que muestre el estado del sistema.

4 Fase 1: Planteamiento del proyecto

Fecha de entrega: 24 de septiembre de 2026

El objetivo de la primera fase es que el equipo defina con claridad qué aplicación van a desarrollar durante el semestre y reciba retroalimentación temprana que les sirva como guía para las siguientes semanas.

1. **Repositorio de GitHub**

- El nombre del repositorio debe ser representativo del proyecto.
- `README.md` que incluya una descripción breve del proyecto que van a desarrollar.
- Todos los integrantes del equipo deben ser colaboradores.

2. **Demo en video**

- No aplica.

3. **Reporte en PDF**

- Link al repositorio de GitHub.
- Describir la aplicación que se va a desarrollar y el problema que resuelve, debe responder por qué una aplicación en la nube es la solución adecuada.
- Lista de los 3 flujos end-to-end que se van a desarrollar.
- Lista de los 5 servicios de AWS que se van a usar y por qué.
- Los roles de cada integrante del equipo.
- Boceto del diagrama de arquitectura.
- Se espera una visión clara de lo que van a entregar al final del semestre.
- Máximo 3 páginas.

Criterio	Descripción	Puntos
Repositorio de GitHub	README con descripción del proyecto. Todos los integrantes son colaboradores.	10
Demo en video	No aplica	0
Reporte en PDF	Describe el problema y por qué la nube. Describe la aplicación, los 3 flujos end-to-end, el boceto de arquitectura, los roles de los integrantes del equipo y la justificación específica de los 5 servicios de AWS.	90

5 Fase 2: Entrega de avances

Fecha de entrega: 15 de octubre de 2026

1. **Repositorio de GitHub**

- Implementación del proyecto funcionando en la nube. Al menos el 80% de la aplicación debe estar corriendo en la nube.
- Scripts en bash o terraform, con instrucciones para crear la infraestructura y desplegar la aplicación.
- El `README.md` y otra documentación debe estar actualizada.
- Commits de todos los integrantes del equipo.

2. **Demo en video**

- Demostración de la aplicación.
- Se debe mencionar qué componentes están corriendo local y cuáles en la nube, y también listar lo que falta por implementar.

3. **Reporte en PDF**

- Link al repositorio de GitHub y al demo en video.
- Descripción actualizada de la aplicación y el problema que resuelve.
- Lista actualizada de los 3 flujos que se van a demostrar.
- Lista actualizada de los servicios de AWS que se utilizan y por qué, en especial si hay algún cambio.
- Descripción de las decisiones técnicas que se tomaron y su justificación.
- Diagrama de la arquitectura.
- Cuáles componentes del proyecto están funcionando en la nube y cuáles local.
- Máximo 4 páginas.

Criterio	Descripción	Puntos
Repositorio de GitHub	Todos los integrantes del equipo están colaborando. El repositorio está bien organizado, contiene el código de la aplicación que se está trabajando completo y coherente con el demo y el reporte. Incluye los scripts para provisionar infraestructura y la documentación necesaria.	20
Demo en video	Se demuestra el 80% de la aplicación funcionando en la nube y hay coherencia con el repositorio de GitHub.	70
Reporte en PDF	Incluye la descripción actualizada del problema y la aplicación, los 3 flujos end-to-end, el diagrama de arquitectura, los roles de los integrantes del equipo y los 5 servicios de AWS. Incluye las decisiones técnicas que se tomaron y su justificación. También incluye un resumen del avance del proyecto, qué componentes son locales, cuáles están en la nube y qué falta por desarrollar.	10

6 Sesión de trabajo en clase

Fecha: 12 de noviembre de 2026

Esta sesión no tiene entregable y no se califica, sin embargo funciona como punto de revisión para que los alumnos muestren el progreso a la maestra, reciban retroalimentación o resuelvan dudas.

En este momento la aplicación debe estar 100% funcionando en la nube, ya deben existir pruebas unitarias y los flujos de CI/CD deben estar implementados. Las semanas entre esta sesión y la Fase 3 se usan para instrumentar el código, agregar métricas y alertas, construir el dashboard y pulir CI/CD.

1. **Repositorio de GitHub**

- Incluye la implementación de la aplicación, la cual debe estar funcionando completamente en la nube.
- El repositorio tiene las instrucciones y los scripts en bash o terraform para provisionar la infraestructura, para desplegar la aplicación y configurarla. Es decir, alguien que no está familiarizado con el proyecto puede desplegar a partir del repositorio.
- Incluye pruebas unitarias para validar el código de la aplicación.
- Incluye los workflows de GitHub Actions para el pipeline de CI/CD.

2. **Demo en video**

- El equipo hace una planeación de cómo van a explicar su proyecto en la presentación final.

3. **Reporte en PDF**

- El equipo actualiza el reporte con los cambios más recientes.

7 Fase 3: Presentación final

- Equipos 1 a 4: 30 de noviembre de 2026
- Equipos 5 a 8: 3 de diciembre de 2026

1. **Repositorio de GitHub**

- El código completo de la aplicación, sus pruebas unitarias y los scripts para provisionar infraestructura, desplegar la aplicación y configurar métricas, alertas y monitoreo.
- Pipelines de CI/CD y Pull Requests demostrando la funcionalidad.

2. **Demo en vivo en el salón de clase (20 minutos)**

- Explicar la problemática, la solución propuesta y describir técnicamente la aplicación.
- Demostrar en vivo los 3 flujos del proyecto, el uso de los 5 servicios de AWS, el funcionamiento de CI/CD, el dashboard de monitoreo, métricas y alertas. El equipo es responsable de que el demo sea 100% funcional.
- Si el sistema falla durante alguna prueba, el equipo debe explicar la causa con precisión para evitar penalizaciones; el sistema debe estar mayormente desplegado y funcionando en la nube.
- Cualquier integrante del equipo debe poder responder preguntas sobre cualquier aspecto de la entrega.

3. **Reporte en PDF**

- Link al repositorio de GitHub y descripción de la aplicación y el problema que resuelve.
- Lista de los 3 flujos desarrollados y los servicios de AWS utilizados y por qué.
- Descripción de los aspectos de CI/CD y monitoreo, y de las decisiones técnicas tomadas y su justificación.
- Diagrama de la arquitectura.
- Máximo 5 páginas.

Criterio	Descripción	Puntos
Repositorio de GitHub	Todos los integrantes del equipo están colaborando. El repositorio está bien organizado, contiene el código de la aplicación que se está trabajando completo y coherente con el demo y el reporte. Incluye los scripts para provisionar infraestructura y la documentación necesaria.	20
Demo en vivo	Se demuestra el 100% de la aplicación funcionando en la nube y hay coherencia con el repositorio de GitHub y el reporte. Se demuestra el flujo de CI/CD, el dashboard de monitoreo, métricas y alertas.	70
Reporte en PDF	Incluye la descripción del problema y la aplicación, los 3 flujos end-to-end, el diagrama de arquitectura, los roles de los integrantes del equipo y los 5 servicios de AWS. Incluye las decisiones técnicas que se tomaron y su justificación.	10

8 Ponderación

El proyecto integrador aporta 30% de su calificación final de la materia

Actividad	Ponderación
Tareas (4)	10%
Prácticas (4 a 5)	30%
Exámenes Prácticos (2)	30%
Proyecto Final	**30%**

La calificación del proyecto integrador se compone de la siguiente forma

Proyecto Final	Ponderación
Fase 1: Planteamiento del proyecto	10%
Fase 2: Entrega de avances	45%
Fase 3: Presentación final	45%